

Comparative Evaluation of Machine Learning Models for Credit Default Prediction

Name: Aryan Vora

Programme: MSc Applied Statistics

University: Birkbeck, University of London

Academic Year: 2025-2026

Word Count: 3000

1. Introduction

Credit risk modelling plays a crucial role in the financial industry, particularly for banks and lending institutions that aim to minimise potential losses arising from customer defaults. Accurately identifying high risk borrowers allows institutions to make informed lending decisions, optimise portfolio performance, and comply with regulatory requirements.

In recent years, the increasing availability of large-scale financial datasets and advancements in computational techniques have led to the widespread adoption of machine learning methods in credit risk analysis. Traditional statistical approaches, such as logistic regression, continue to be widely used due to their interpretability and simplicity. However, more advanced techniques, including ensemble methods and neural networks, have been introduced to capture complex, non-linear relationships within the data.

The primary objective of this project is to develop and compare multiple machine learning models for predicting credit default. Specifically, this study evaluates the performance of logistic regression, random forest, and artificial neural networks using a real world credit dataset. The models are assessed based on key

performance metrics such as accuracy, precision, recall, and ROC-AUC, with particular emphasis on the ability to correctly identify defaulters.

This analysis is conducted from the perspective of a financial practitioner, where the cost of misclassification is asymmetric. In particular, failing to identify a defaulter (false negative) can lead to significant financial losses, making recall an important metric in model evaluation. Therefore, this study not only compares predictive performance but also examines the practical implications of each model in a credit risk context.

The findings of this project aim to provide insights into the trade-offs between model complexity, interpretability, and predictive performance, thereby supporting more effective decision making in credit risk management.

2. Background

Credit risk modelling is a fundamental component of modern financial systems, used by banks and lending institutions to assess the likelihood that a borrower will default on their financial obligations. Traditional credit scoring techniques have largely relied on statistical models, particularly logistic regression, due to their interpretability and ease of implementation. Logistic regression estimates the probability of default by modelling the relationship between a binary response variable and a set of explanatory variables, making it suitable for classification problems in finance.

However, financial data often exhibit complex, non-linear relationships that cannot be fully captured by linear models. This limitation has led to the adoption of machine learning techniques, which offer greater flexibility in modelling interactions and non-linear patterns. Among these, ensemble methods such as Random Forest have gained popularity due to

their ability to improve predictive performance by combining multiple decision trees. Random Forest reduces overfitting and handles multicollinearity effectively, making it well suited for high dimensional financial datasets.

This study builds on these concepts by applying and comparing logistic regression, Random Forest, and neural networks to a real world credit dataset, with the aim of evaluating their effectiveness in predicting default and their relevance in a financial practitioner setting.

```
In [1]: # Importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_
from sklearn.neural_network import MLPClassifier

# Display Settings
pd.set_option('display.max_columns', None)
```

3. Data

The dataset used in this study is the “Default of Credit Card Clients” dataset obtained from the UCI Machine Learning Repository. The dataset contains information on 30,000 credit card clients and includes demographic, financial, and behavioural variables.

The response variable is a binary indicator of default payment, where 1 represents default and 0 represents non-default. The dataset includes 23 explanatory variables capturing different aspects of customer behaviour and financial status.

```
In [2]: df = pd.read_csv("Data - Data.csv", header = 1)

df.head()
```

Out[2]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4
0	1	20000	2	2	1	24	2	2	-1	-1
1	2	120000	2	2	2	26	-1	2	0	0
2	3	90000	2	2	2	34	0	0	0	0
3	4	50000	2	2	1	37	0	0	0	0
4	5	50000	1	2	1	57	-1	0	-1	0

```
In [3]: df.tail()
```

Out[3]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3
29995	29996	220000	1	3	1	39	0	0	0
29996	29997	150000	1	3	2	43	-1	-1	-1
29997	29998	30000	1	2	2	37	4	3	2
29998	29999	80000	1	3	1	41	1	-1	0
29999	30000	50000	1	2	1	46	0	0	0

3.1 Variable Description

The explanatory variables can be grouped into four main categories:

- **Credit Information:** LIMIT_BAL represents the total credit limit assigned to the customer.
- **Demographic Variables:** SEX, EDUCATION, MARRIAGE, and AGE provide personal information about the customer.
- **Repayment History:** PAY_0 to PAY_6 represent the customer's repayment status over the past six months. These variables are ordinal, where -1 indicates timely payment and positive values indicate the number of months of payment delay.

- **Billing and Payment Behaviour:** BILL_AMT1 to BILL_AMT6 represent the monthly bill statements, while PAY_AMT1 to PAY_AMT6 represent the amounts paid by the customer over the same period.

The repayment history variables are particularly important, as they capture past financial behaviour, which is a strong indicator of future default risk.

```
In [4]: # Basic Structure
df.shape
```

```
Out[4]: (30000, 25)
```

```
In [5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ID                                    30000 non-null  int64
1   LIMIT_BAL                            30000 non-null  int64
2   SEX                                  30000 non-null  int64
3   EDUCATION                           30000 non-null  int64
4   MARRIAGE                             30000 non-null  int64
5   AGE                                  30000 non-null  int64
6   PAY_0                                30000 non-null  int64
7   PAY_2                                30000 non-null  int64
8   PAY_3                                30000 non-null  int64
9   PAY_4                                30000 non-null  int64
10  PAY_5                                30000 non-null  int64
11  PAY_6                                30000 non-null  int64
12  BILL_AMT1                            30000 non-null  int64
13  BILL_AMT2                            30000 non-null  int64
14  BILL_AMT3                            30000 non-null  int64
15  BILL_AMT4                            30000 non-null  int64
16  BILL_AMT5                            30000 non-null  int64
17  BILL_AMT6                            30000 non-null  int64
18  PAY_AMT1                             30000 non-null  int64
19  PAY_AMT2                             30000 non-null  int64
20  PAY_AMT3                             30000 non-null  int64
21  PAY_AMT4                             30000 non-null  int64
22  PAY_AMT5                             30000 non-null  int64
23  PAY_AMT6                             30000 non-null  int64
24  default payment next month           30000 non-null  int64
dtypes: int64(25)
memory usage: 5.7 MB
```

```
In [6]: df.describe()
```

Out[6]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	
count	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30
mean	15000.500000	167484.322667	1.603733	1.853133	1.551867	
std	8660.398374	129747.661567	0.489129	0.790349	0.521970	
min	1.000000	10000.000000	1.000000	0.000000	0.000000	
25%	7500.750000	50000.000000	1.000000	1.000000	1.000000	
50%	15000.500000	140000.000000	2.000000	2.000000	2.000000	
75%	22500.250000	240000.000000	2.000000	2.000000	2.000000	
max	30000.000000	1000000.000000	2.000000	6.000000	3.000000	

```
In [7]: # Rename Target Variable
df.rename(columns={"default payment next month": "default"}, inplace=True)
```

```
In [8]: # Check Missing Values
df.isnull().sum()
```

```
Out[8]: ID          0
LIMIT_BAL  0
SEX         0
EDUCATION   0
MARRIAGE    0
AGE         0
PAY_0       0
PAY_2       0
PAY_3       0
PAY_4       0
PAY_5       0
PAY_6       0
BILL_AMT1   0
BILL_AMT2   0
BILL_AMT3   0
BILL_AMT4   0
BILL_AMT5   0
BILL_AMT6   0
PAY_AMT1    0
PAY_AMT2    0
PAY_AMT3    0
PAY_AMT4    0
PAY_AMT5    0
PAY_AMT6    0
default     0
dtype: int64
```

```
In [9]: df.isna().sum()
```

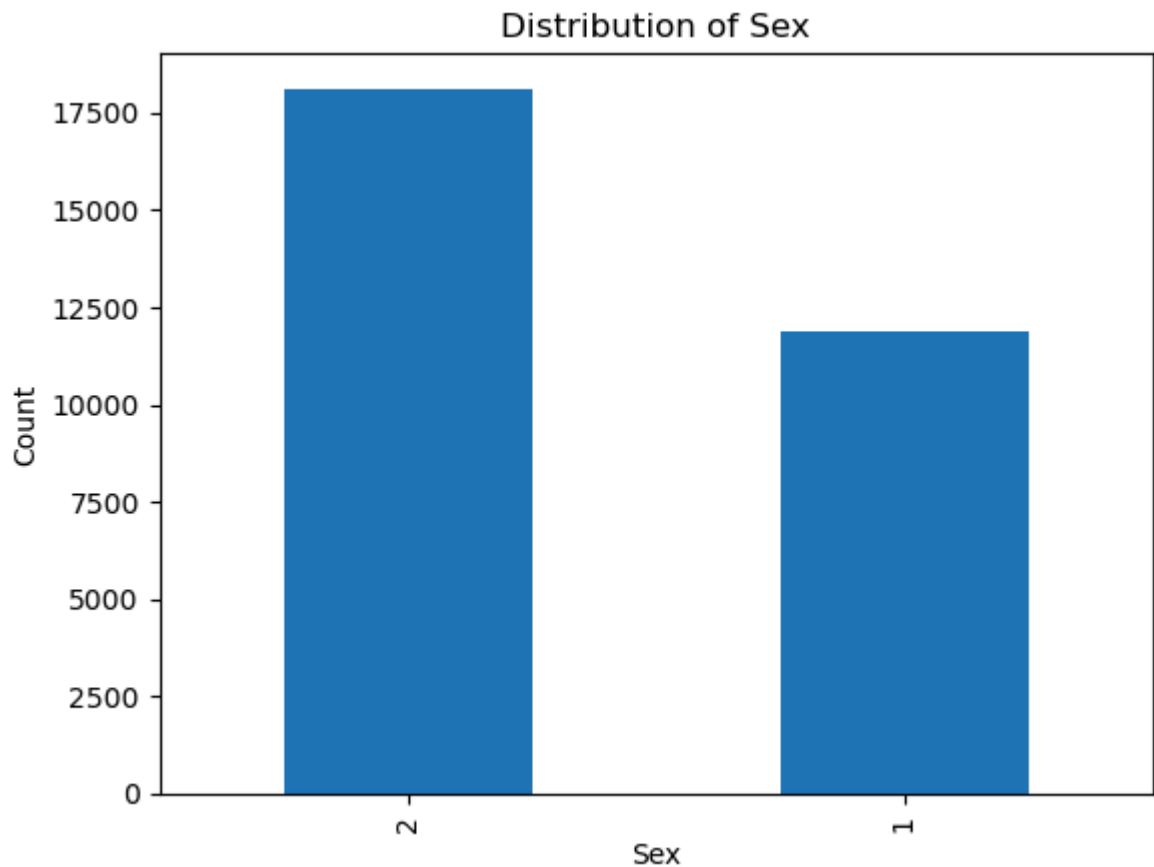
```
Out[9]: ID          0
        LIMIT_BAL   0
        SEX         0
        EDUCATION   0
        MARRIAGE     0
        AGE         0
        PAY_0        0
        PAY_2        0
        PAY_3        0
        PAY_4        0
        PAY_5        0
        PAY_6        0
        BILL_AMT1    0
        BILL_AMT2    0
        BILL_AMT3    0
        BILL_AMT4    0
        BILL_AMT5    0
        BILL_AMT6    0
        PAY_AMT1     0
        PAY_AMT2     0
        PAY_AMT3     0
        PAY_AMT4     0
        PAY_AMT5     0
        PAY_AMT6     0
        default      0
        dtype: int64
```

```
In [10]: # Understand Categorical Variables
        # Gender
        df['SEX'].value_counts()
```

```
Out[10]: SEX
         2    18112
         1    11888
        Name: count, dtype: int64

X2: Gender (1 = male; 2 = female)
```

```
In [11]: df["SEX"].value_counts().plot(kind='bar')
        plt.title(f"Distribution of Sex")
        plt.xlabel("Sex")
        plt.ylabel("Count")
        plt.show()
```



```
In [12]: # Education
df['EDUCATION'].value_counts()
```

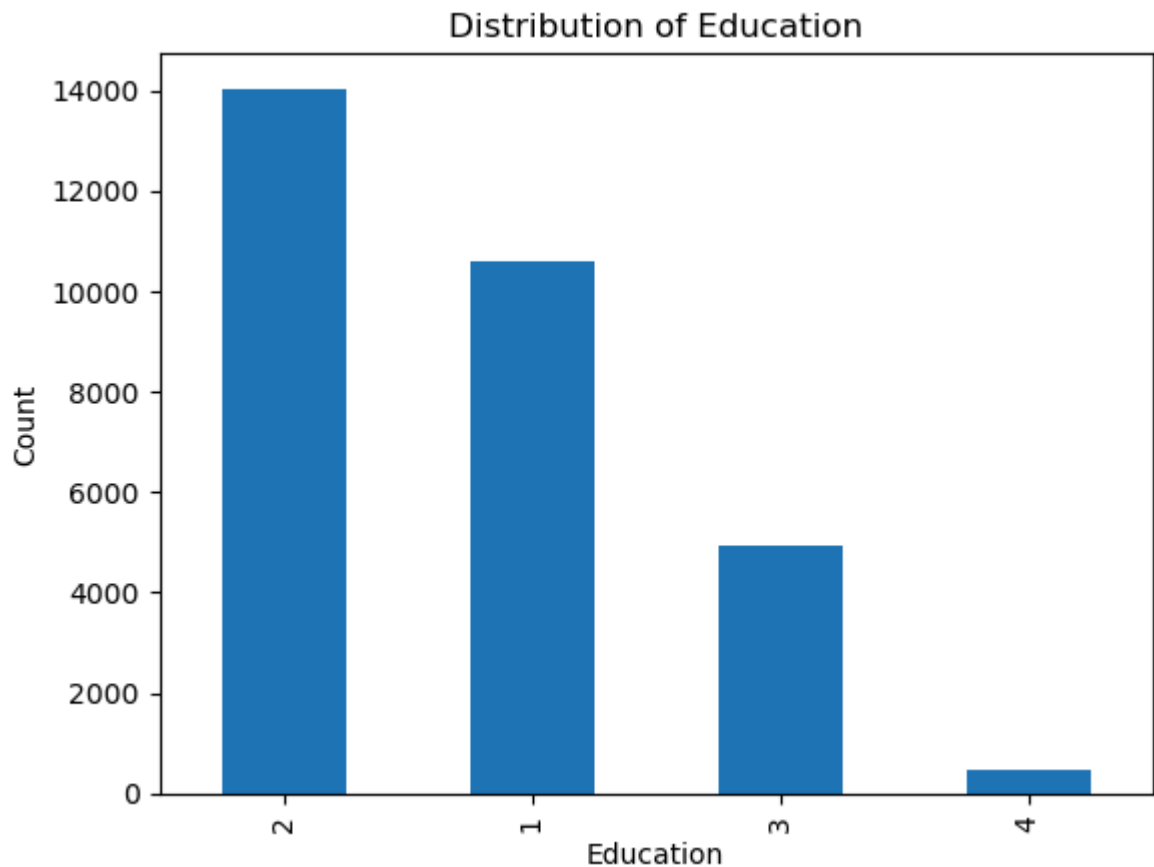
```
Out[12]: EDUCATION
2      14030
1      10585
3       4917
5        280
4        123
6         51
0         14
Name: count, dtype: int64
```

```
In [13]: # Handle Categorical Variables
df['EDUCATION'] = df['EDUCATION'].replace([0,5,6], 4)
df['EDUCATION'].value_counts()
```

```
Out[13]: EDUCATION
2      14030
1      10585
3       4917
4        468
Name: count, dtype: int64
```

X3: Education (1 = graduate school; 2 = university; 3 = high school; 4 = others)

```
In [14]: df["EDUCATION"].value_counts().plot(kind='bar')
plt.title(f"Distribution of Education")
plt.xlabel("Education")
plt.ylabel("Count")
plt.show()
```



```
In [15]: # Marriage
df['MARRIAGE'].value_counts()
```

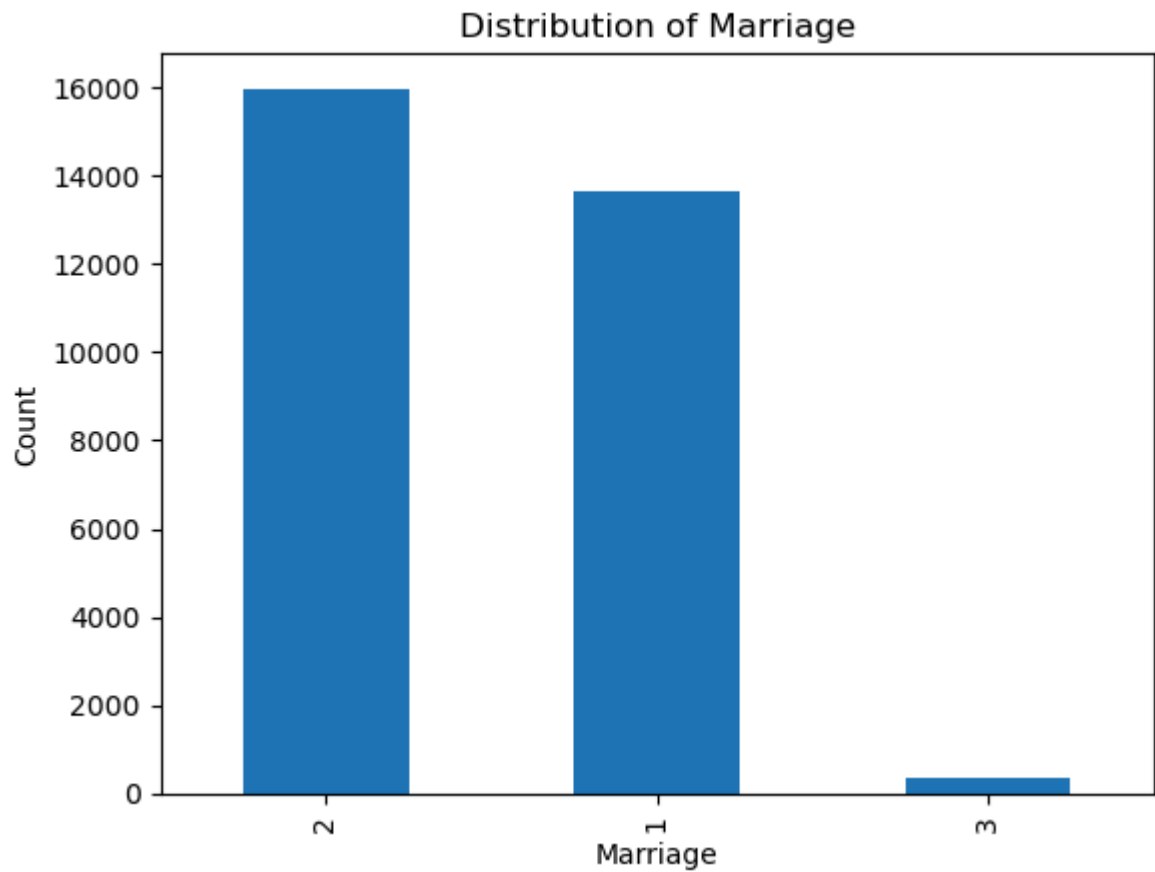
```
Out[15]: MARRIAGE
2      15964
1      13659
3         323
0          54
Name: count, dtype: int64
```

```
In [16]: # Handle Categorical Variables
df['MARRIAGE'] = df['MARRIAGE'].replace(0, 3)
df['MARRIAGE'].value_counts()
```

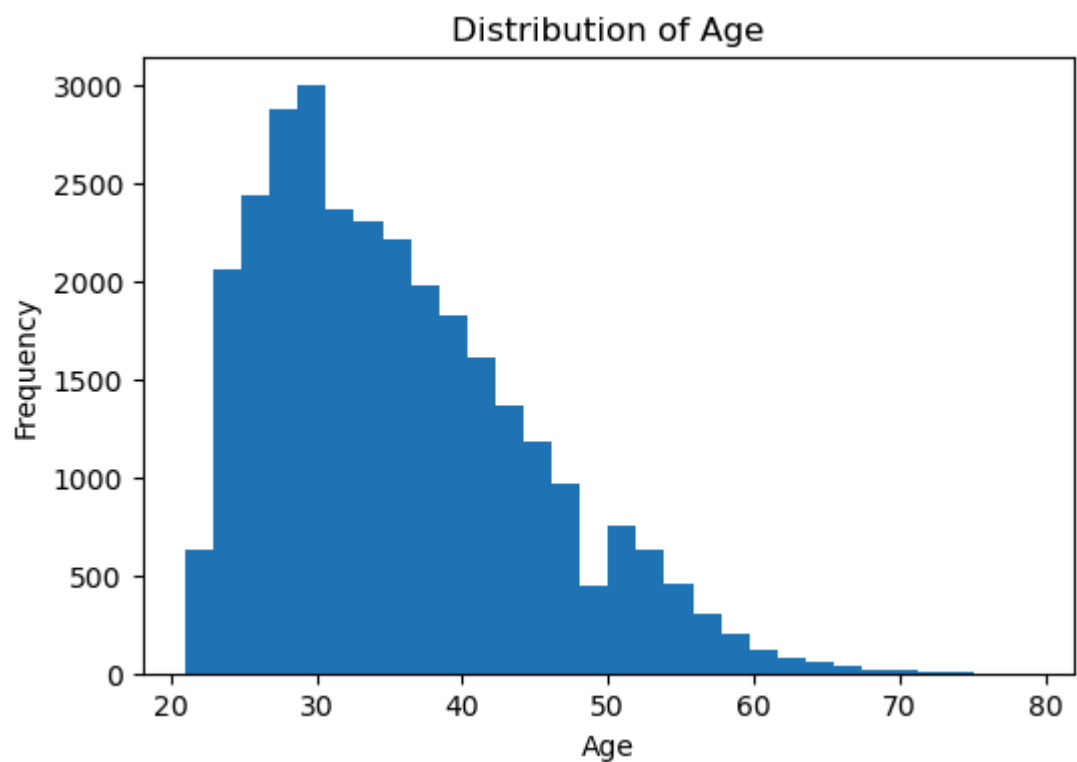
```
Out[16]: MARRIAGE
2      15964
1      13659
3         377
Name: count, dtype: int64
```

X4: Marital status (1 = married; 2 = single; 3 = others)

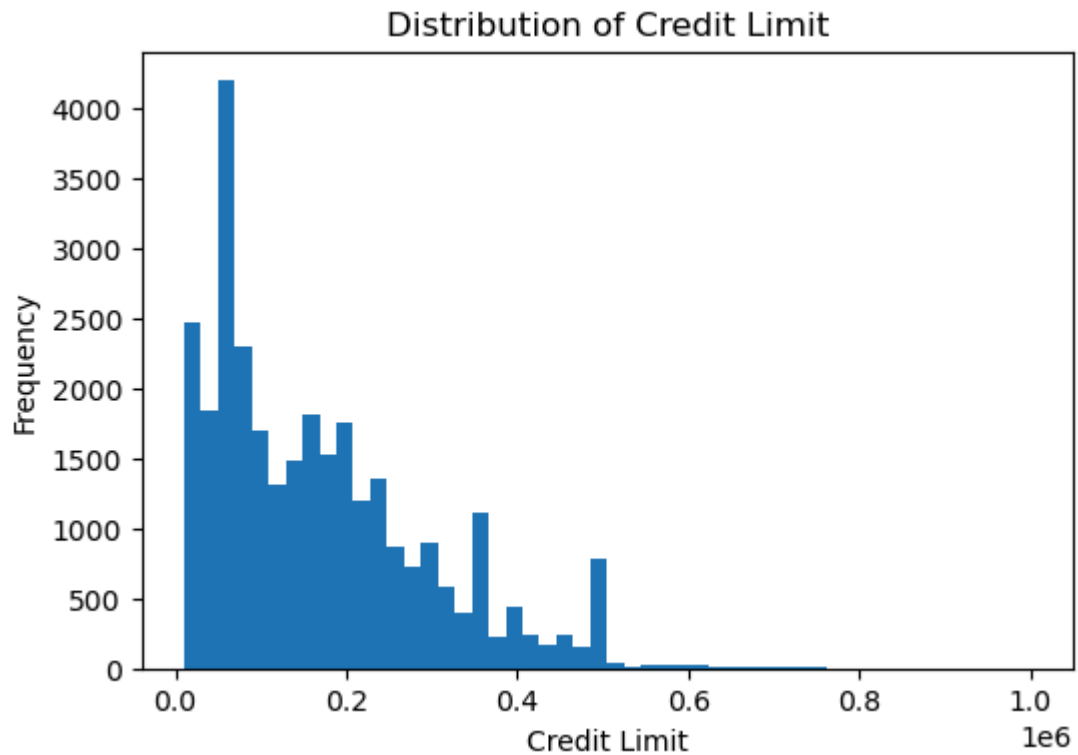
```
In [17]: df["MARRIAGE"].value_counts().plot(kind='bar')
plt.title(f"Distribution of Marriage")
plt.xlabel("Marriage")
plt.ylabel("Count")
plt.show()
```



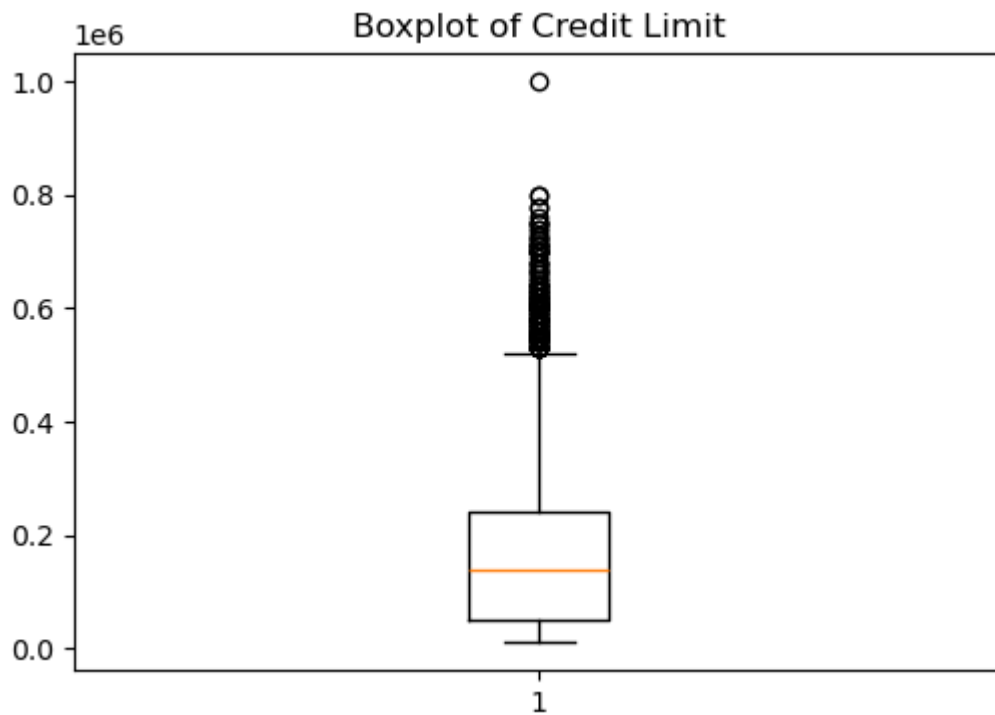
```
In [18]: # Univariate Analysis
# Age
plt.figure(figsize=(6,4))
plt.hist(df['AGE'], bins=30)
plt.title("Distribution of Age")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



```
In [19]: # Credit Limit
plt.figure(figsize=(6,4))
plt.hist(df['LIMIT_BAL'], bins=50)
plt.title("Distribution of Credit Limit")
plt.xlabel("Credit Limit")
plt.ylabel("Frequency")
plt.show()
```

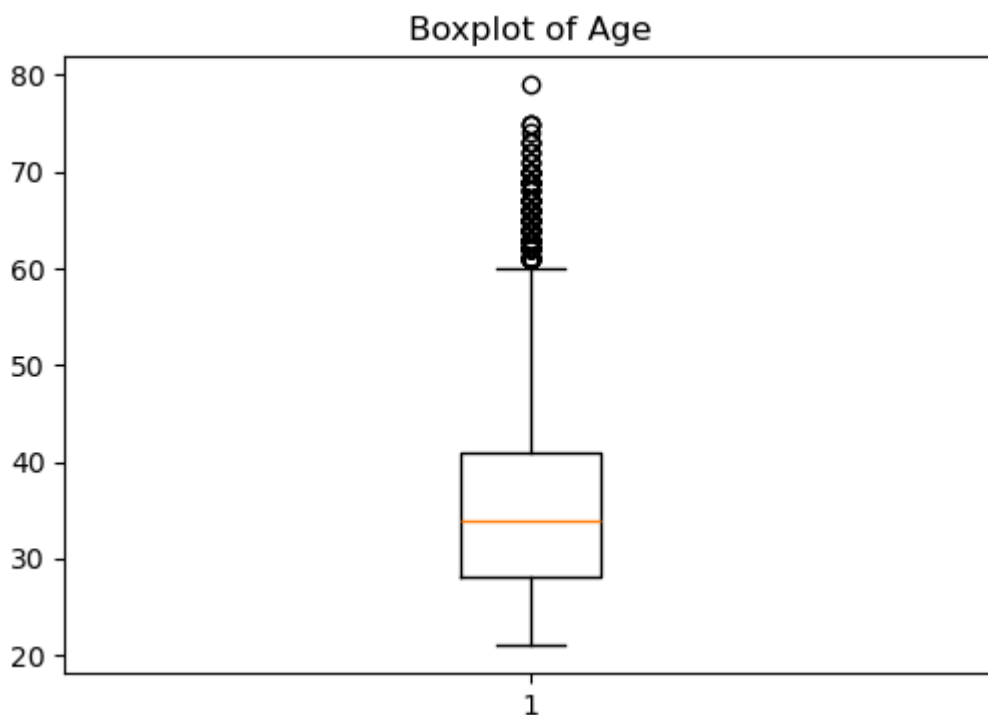


```
In [20]: # Outlier Detection
# Credit Limit
plt.figure(figsize=(6,4))
plt.boxplot(df['LIMIT_BAL'])
plt.title("Boxplot of Credit Limit")
plt.show()
```



The boxplot reveal the presence of outliers in credit limit, which is expected in financial datasets due to high-value customers.

```
In [21]: # Age
plt.figure(figsize=(6,4))
plt.boxplot(df['AGE'])
plt.title("Boxplot of Age")
plt.show()
```



The boxplot of age indicates that the majority of customers are between 28 and 40 years old, with a median around 34 years.

While several observations above 60 years are identified as outliers. These values are realistic and reflect the presence of older customers in the dataset. Therefore, no outlier treatment was applied to the age variable.

```
In [22]: # Skewness  
df[['LIMIT_BAL', 'AGE']].skew()
```

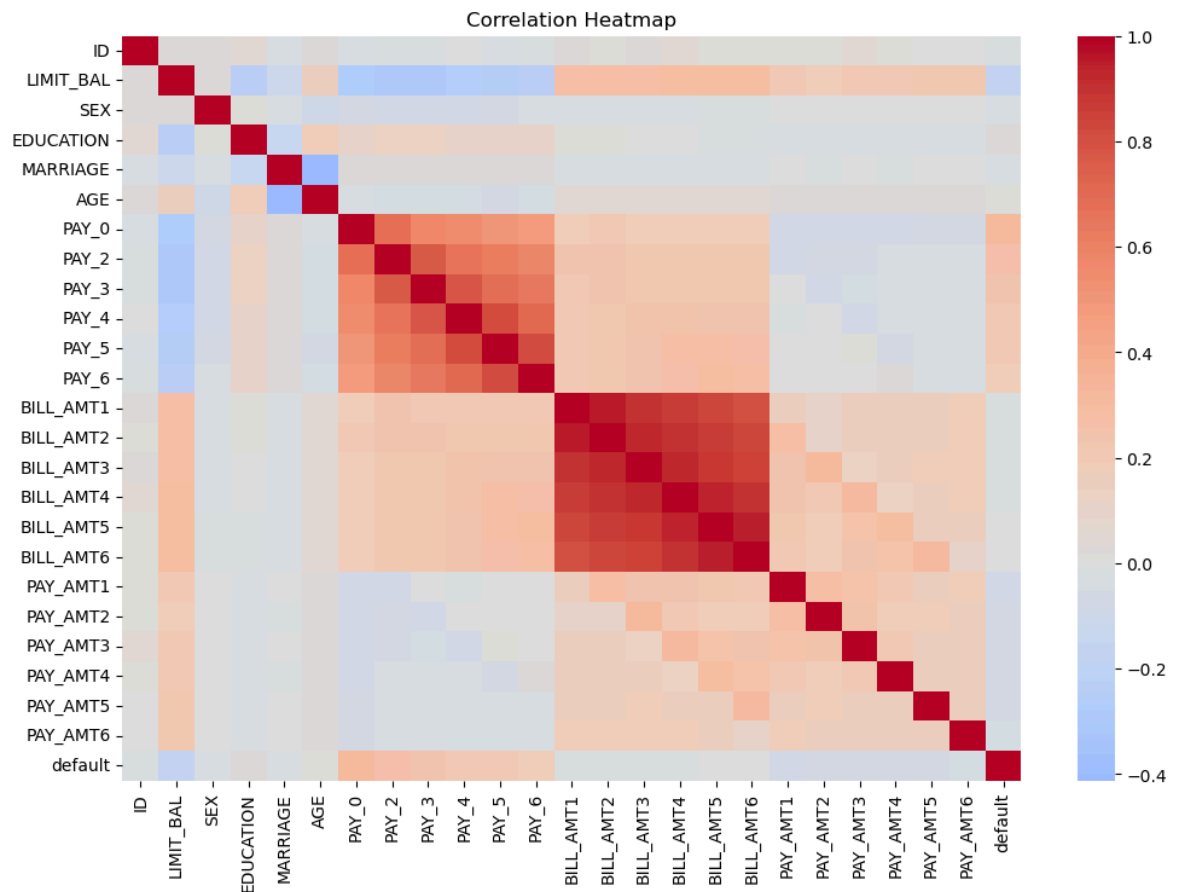
```
Out[22]: LIMIT_BAL    0.992867  
AGE          0.732246  
dtype: float64
```

```
In [23]: # Correlation Analysis  
# Correlation Matrix  
corr_matrix = df.corr()  
corr_matrix
```

Out[23]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	
ID	1.000000	0.026179	0.018497	0.037659	-0.028208	0.018678	-0
LIMIT_BAL	0.026179	1.000000	0.024755	-0.231088	-0.111012	0.144713	-0
SEX	0.018497	0.024755	1.000000	0.013627	-0.028522	-0.090874	-0
EDUCATION	0.037659	-0.231088	0.013627	1.000000	-0.136797	0.182434	0
MARRIAGE	-0.028208	-0.111012	-0.028522	-0.136797	1.000000	-0.412001	0
AGE	0.018678	0.144713	-0.090874	0.182434	-0.412001	1.000000	-0
PAY_0	-0.030575	-0.271214	-0.057643	0.112593	0.018557	-0.039447	1
PAY_2	-0.011215	-0.296382	-0.070771	0.129703	0.023620	-0.050148	0
PAY_3	-0.018494	-0.286123	-0.066096	0.122425	0.032399	-0.053048	0
PAY_4	-0.002735	-0.267460	-0.060173	0.116531	0.031831	-0.049722	0
PAY_5	-0.022199	-0.249411	-0.055064	0.104088	0.034377	-0.053826	0
PAY_6	-0.020270	-0.235195	-0.044008	0.088986	0.033168	-0.048773	0
BILL_AMT1	0.019389	0.285430	-0.033642	0.016597	-0.027832	0.056239	0
BILL_AMT2	0.017982	0.278314	-0.031183	0.011980	-0.025294	0.054283	0
BILL_AMT3	0.024354	0.283236	-0.024563	0.006714	-0.029082	0.053710	0
BILL_AMT4	0.040351	0.293988	-0.021880	-0.006131	-0.027274	0.051353	0
BILL_AMT5	0.016705	0.295562	-0.017005	-0.012439	-0.029270	0.049345	0
BILL_AMT6	0.016730	0.290389	-0.016733	-0.012646	-0.025066	0.047613	0
PAY_AMT1	0.009742	0.195236	-0.000242	-0.041088	-0.004653	0.026147	-0
PAY_AMT2	0.008406	0.178408	-0.001391	-0.032793	-0.009513	0.021785	-0
PAY_AMT3	0.039151	0.210167	-0.008597	-0.044293	-0.004250	0.029247	-0
PAY_AMT4	0.007793	0.203242	-0.002229	-0.040949	-0.013970	0.021379	-0
PAY_AMT5	0.000652	0.217202	-0.001667	-0.045138	-0.003019	0.022850	-0
PAY_AMT6	0.003000	0.219595	-0.002766	-0.044061	-0.008383	0.019478	-0
default	-0.013952	-0.153520	-0.039961	0.033842	-0.027575	0.013890	0

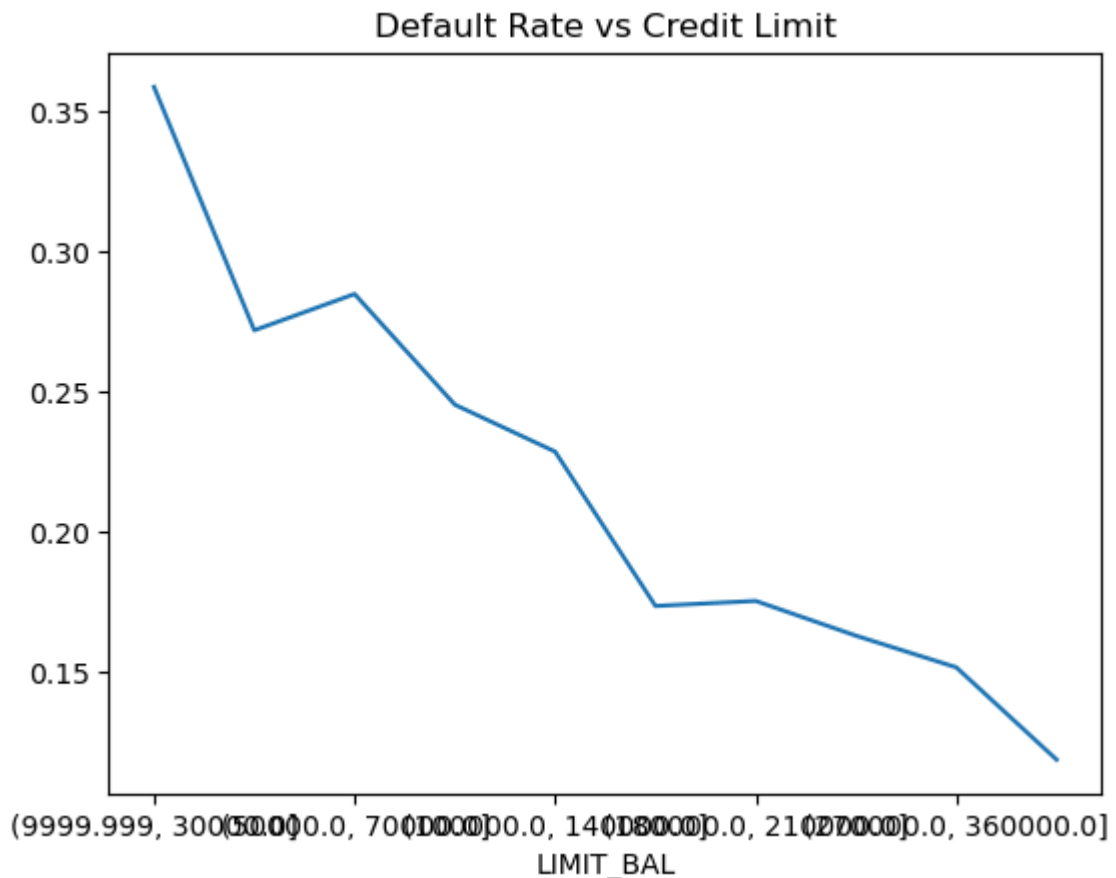
```
In [24]: # Heatmap
plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, cmap='coolwarm', center=0)
plt.title("Correlation Heatmap")
plt.show()
```



```
In [25]: # Linearity
df.groupby(pd.qcut(df['LIMIT_BAL'], 10))['default'].mean().plot()
plt.title("Default Rate vs Credit Limit")
plt.show()
```

/var/folders/wm/9x5zvmqj35q7jk6f_dx487vw0000gn/T/ipykernel_65007/3762106691.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
df.groupby(pd.qcut(df['LIMIT_BAL'], 10))['default'].mean().plot()
```



The relationship between credit limit and default probability shows a clear decreasing trend, where higher credit limits are associated with lower default rates. This suggests that credit limit is a strong predictor of default risk. The observed pattern appears approximately monotonic, supporting the assumption of linearity in the log-odds for this variable.

```
In [26]: # Feature Selection
df.drop(columns=['ID'], inplace=True)

In [27]: # Splitting Dataset
X = df.drop('default', axis=1)
y = df['default']

In [28]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
# Stratify is important due to class imbalance

In [29]: # Encoding Categorical Variables
X_train = pd.get_dummies(X_train, columns=['SEX', 'EDUCATION', 'MARRIAGE'],
X_test = pd.get_dummies(X_test, columns=['SEX', 'EDUCATION', 'MARRIAGE'], d
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0

In [30]: # Feature Scaling
col = ['LIMIT_BAL', 'AGE', 'PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5',
        'PAY_6', 'BILL_AMT1', 'BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4',
        'BILL_AMT5', 'BILL_AMT6', 'PAY_AMT1', 'PAY_AMT2', 'PAY_AMT3',
        'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6', 'SEX_2', 'EDUCATION_2',
```

```
'EDUCATION_3', 'EDUCATION_4', 'MARRIAGE_2', 'MARRIAGE_3']  
num_cols = ['LIMIT_BAL', 'AGE'] + [col for col in X_train.columns if 'BILL'  
scaler = StandardScaler()  
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])  
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

3.2 Data Quality and Preprocessing

Initial data exploration showed that the dataset does not contain missing values, ensuring completeness of the observations. The ID variable was removed as it does not contribute to predictive modelling. Categorical variables such as gender, education, and marital status were encoded appropriately for model implementation.

Certain variables, particularly billing amounts and payment amounts, exhibited large differences in scale and skewed distributions. These variables were standardised using feature scaling techniques to improve model performance, especially for logistic regression and neural networks. The repayment status variables, although ordinal, were treated as numerical features to preserve the ordering information.

3.3 Exploratory Data Analysis

Exploratory analysis revealed that the dataset is moderately imbalanced, with a significantly higher proportion of non-default cases compared to default cases. This imbalance highlights the importance of using evaluation metrics beyond accuracy.

The distribution of credit limits is right-skewed, indicating that most customers have relatively lower credit limits, with a small number of high value clients. Age distribution shows that the majority of customers fall within the working age population.

Correlation analysis indicates strong relationships among billing and payment variables, suggesting the presence of multicollinearity. Additionally, repayment history variables show a noticeable association with default behaviour, confirming their importance in predictive modelling.

3.4 Data Splitting

The dataset was split into training and testing sets using a 70:30 ratio, with stratification applied to preserve the class distribution of the target variable. This ensures that model evaluation is performed on unseen data while maintaining representativeness of the dataset.

4. Methodology

This study adopts a comparative modelling approach to evaluate the performance of different machine learning techniques in predicting credit default. Three models were implemented: Logistic Regression, Random Forest, and Artificial Neural Networks. These models were selected to represent increasing levels of complexity, allowing for a comprehensive comparison between traditional statistical methods and modern machine learning approaches.

4.1 Modelling Framework

The modelling process follows a structured pipeline. The dataset was first split into training and testing sets using a 70:30 ratio with stratification to preserve class distribution. Categorical variables were encoded, and feature scaling was applied where required. Logistic regression and neural network models were trained on scaled data, while Random Forest was trained on unscaled data due to its tree-based structure.

Model performance was evaluated using multiple metrics, including accuracy, precision, recall, and the area under the ROC curve (ROC-AUC). Given the imbalanced nature of the dataset, particular emphasis was placed on recall for the default class and ROC-AUC, as these metrics provide a more meaningful assessment of model performance in credit risk applications.

4.2 Logistic Regression

Logistic regression was used as the baseline model due to its widespread application in credit risk modelling and its interpretability. The model estimates the probability of default by modelling the log-odds of the response variable as a linear function of the predictors. To address class imbalance, class weights were incorporated to ensure that default cases were appropriately penalised during model training.

Assumptions of Logistic Regression:-

1. Binary Dependent Variable
2. Independence of Observations
3. No Strong Multicollinearity
4. Linearity in the Log-odds
5. No Extreme Outliers
6. Large Sample Size

```
In [31]: # Fitting Logistic Regression Model
model = LogisticRegression(class_weight='balanced', max_iter=1000)
model.fit(X_train, y_train)
```

```
Out[31]: LogisticRegression
LogisticRegression(class_weight='balanced', max_iter=1000)
```

```
In [32]: # Predictions
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1]
```

```
In [33]: # Accuracy
accuracy_score(y_test, y_pred)
```

```
Out[33]: 0.6827777777777778
```

The logistic regression model achieved an accuracy of approximately 68%. However, given the class imbalance in the dataset, accuracy alone is not an appropriate performance metric. Therefore, additional evaluation metrics such as precision, recall, and ROC-AUC were considered for a more comprehensive assessment.

```
In [34]: # Confusion Matrix
confusion_matrix(y_test, y_pred)
```

```
Out[34]: array([[4875, 2134],
               [ 721, 1270]])
```

The confusion matrix shows that the model correctly identified a substantial number of non-defaulters (4875) and defaulters (1270). However, it also produced a significant number of false positives (2134), indicating that many non-defaulters were incorrectly classified as high risk.

```
In [35]: # Classification Report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.87	0.70	0.77	7009
1	0.37	0.64	0.47	1991
accuracy			0.68	9000
macro avg	0.62	0.67	0.62	9000
weighted avg	0.76	0.68	0.71	9000

The classification report indicates that the model performs well in identifying non-defaulters, with a precision of 87% and recall of 70%. For defaulters, the model achieves a recall of approximately 64%, suggesting a reasonable ability to detect risky customers. However, the precision for defaulters is relatively low (37%), indicating a high number of false positives. This reflects a trade-off between identifying risky customers and incorrectly classifying safe customers as high risk.

```
In [36]: # ROC-AUC
roc_auc_score(y_test, y_prob)
```

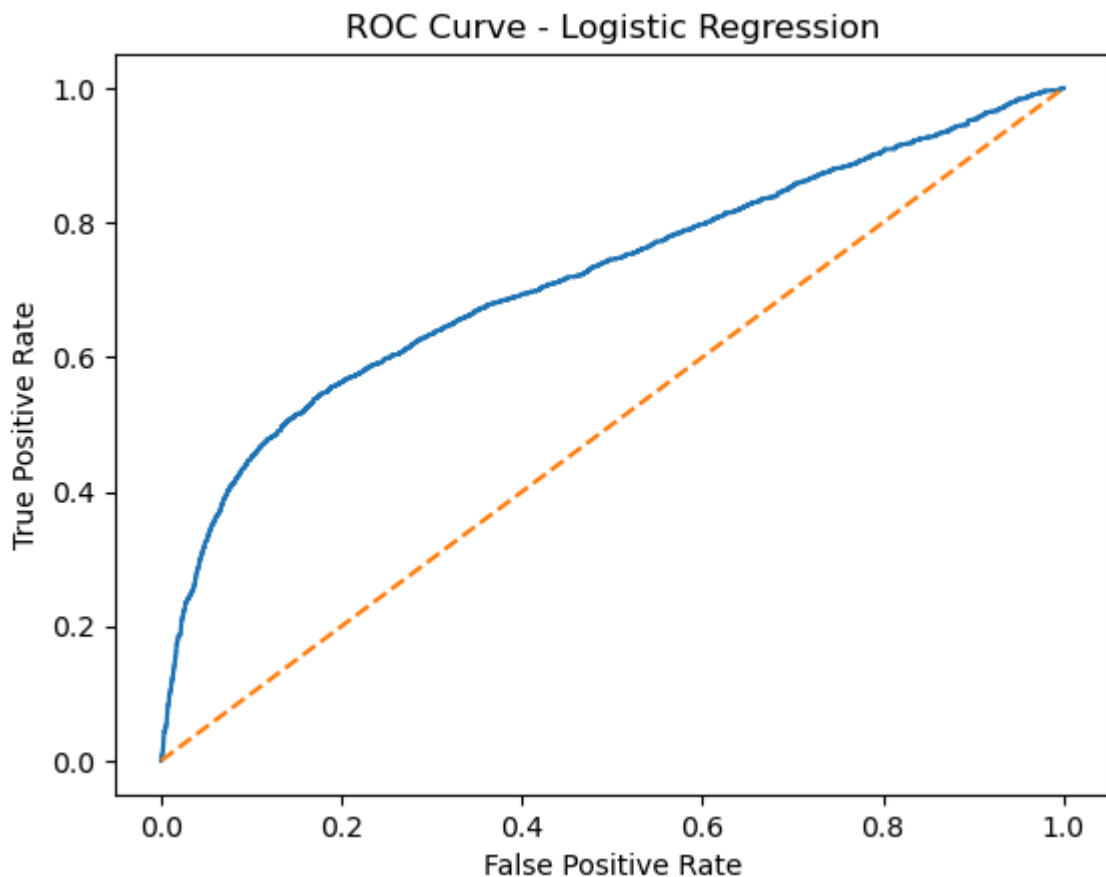
```
Out[36]: np.float64(0.7172911931627837)
```

The logistic regression model achieved a ROC-AUC score of approximately 0.72, indicating a good ability to distinguish between defaulters and non-defaulters. This suggests that, despite moderate accuracy, the model has meaningful predictive power and is effective in ranking customers by risk. The logistic regression model achieved a ROC-AUC score of approximately 0.72, indicating a good ability to distinguish between defaulters and non-defaulters. This suggests that, despite moderate

accuracy, the model has meaningful predictive power and is effective in ranking customers by risk.

```
In [37]: # ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr)
plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Logistic Regression")
plt.show()
```



The ROC curve lies significantly above the diagonal baseline, indicating that the model performs better than random classification. The curve demonstrates a reasonable trade-off between true positive rate and false positive rate, consistent with the ROC-AUC value of approximately 0.72. This suggests that the model has a good ability to distinguish between defaulters and non-defaulters, although there is room for improvement. The ROC curve lies significantly above the diagonal baseline, indicating that the model performs better than random classification. The curve demonstrates a reasonable trade-off

between true positive rate and false positive rate, consistent with the ROC-AUC value of approximately 0.72. This suggests that the model has a good ability to distinguish between defaulters and non-defaulters, although there is room for improvement.

4.3 Random Forest

Random Forest is an ensemble learning method that constructs multiple decision trees and aggregates their predictions. It is particularly effective in capturing non-linear relationships and interactions between variables. Unlike logistic regression, Random Forest does not require feature scaling and is robust to multicollinearity.

Assumptions of Random Forest:-

1. Independent Observations
2. Sufficient Data
3. Handles Multicollinearity
4. Can Capture Non Linearity

```
In [38]: df = pd.read_csv("Data - Data.csv", header = 1)
df.head()
```

Out[38]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4
0	1	20000	2	2	1	24	2	2	-1	-1
1	2	120000	2	2	2	26	-1	2	0	0
2	3	90000	2	2	2	34	0	0	0	0
3	4	50000	2	2	1	37	0	0	0	0
4	5	50000	1	2	1	57	-1	0	-1	0

```
In [39]: # Rename Target Variable
df.rename(columns={"default payment next month": "default"}, inplace=True)
```

```
In [40]: # Handle Categorical Variables
df['EDUCATION'] = df['EDUCATION'].replace([0,5,6], 4)
df['EDUCATION'].value_counts()
```

```
Out[40]: EDUCATION
2      14030
1      10585
3       4917
4        468
Name: count, dtype: int64
```

```
In [41]: # Handle Categorical Variables
df['MARRIAGE'] = df['MARRIAGE'].replace(0, 3)
df['MARRIAGE'].value_counts()
```

```
Out[41]: MARRIAGE
2      15964
1      13659
3        377
Name: count, dtype: int64
```

```
In [42]: # Feature Selection
df.drop(columns=['ID'], inplace=True)
```

```
In [43]: # Splitting Dataset
X = df.drop('default', axis=1)
y = df['default']
```

```
In [44]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
# Stratify is important due to class imbalance
```

```
In [45]: # Encoding Categorical Variables
X_train = pd.get_dummies(X_train, columns=['SEX', 'EDUCATION', 'MARRIAGE'],
X_test = pd.get_dummies(X_test, columns=['SEX', 'EDUCATION', 'MARRIAGE'], d
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0
```

```
In [46]: # Fit Random Forest
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    random_state=42,
    class_weight='balanced'
)
rf_model.fit(X_train, y_train)
```

```
Out[46]: ▼ RandomForestClassifier ⓘ ?
RandomForestClassifier(class_weight='balanced', random_state=42)
```

```
In [47]: # Predictions
y_pred_rf = rf_model.predict(X_test)
y_prob_rf = rf_model.predict_proba(X_test)[:,1]
```

```
In [48]: # Accuracy
accuracy_score(y_test, y_pred_rf)
```

```
Out[48]: 0.8126666666666666
```

The Random Forest model achieved a higher accuracy of approximately 81%, outperforming logistic regression. However,

due to class imbalance, accuracy is not a reliable performance metric.

```
In [49]: # Confusion Matrix
confusion_matrix(y_test, y_pred_rf)
```

```
Out[49]: array([[6638,  371],
               [1315,  676]])
```

The Random Forest model demonstrates strong performance in identifying non-defaulters, with a low number of false positives. However, it exhibits a significantly lower recall for defaulters compared to logistic regression, indicating that many risky customers are not identified. This highlights a trade-off between precision and recall, where Random Forest is more conservative but less effective in detecting default risk.

```
In [50]: # Classification Report
print(classification_report(y_test, y_pred_rf))
```

	precision	recall	f1-score	support
0	0.83	0.95	0.89	7009
1	0.65	0.34	0.45	1991
accuracy			0.81	9000
macro avg	0.74	0.64	0.67	9000
weighted avg	0.79	0.81	0.79	9000

The Random Forest model achieved a higher overall accuracy (81%) compared to logistic regression. It also demonstrated improved precision in identifying defaulters. However, its recall for defaulters was significantly lower (34%), indicating that a large proportion of risky customers were not detected. This highlights a trade-off between precision and recall, where Random Forest is more conservative but less effective for risk identification.

```
In [51]: # ROC-AUC
roc_auc_score(y_test, y_prob_rf)
```

```
Out[51]: np.float64(0.7558104063520541)
```

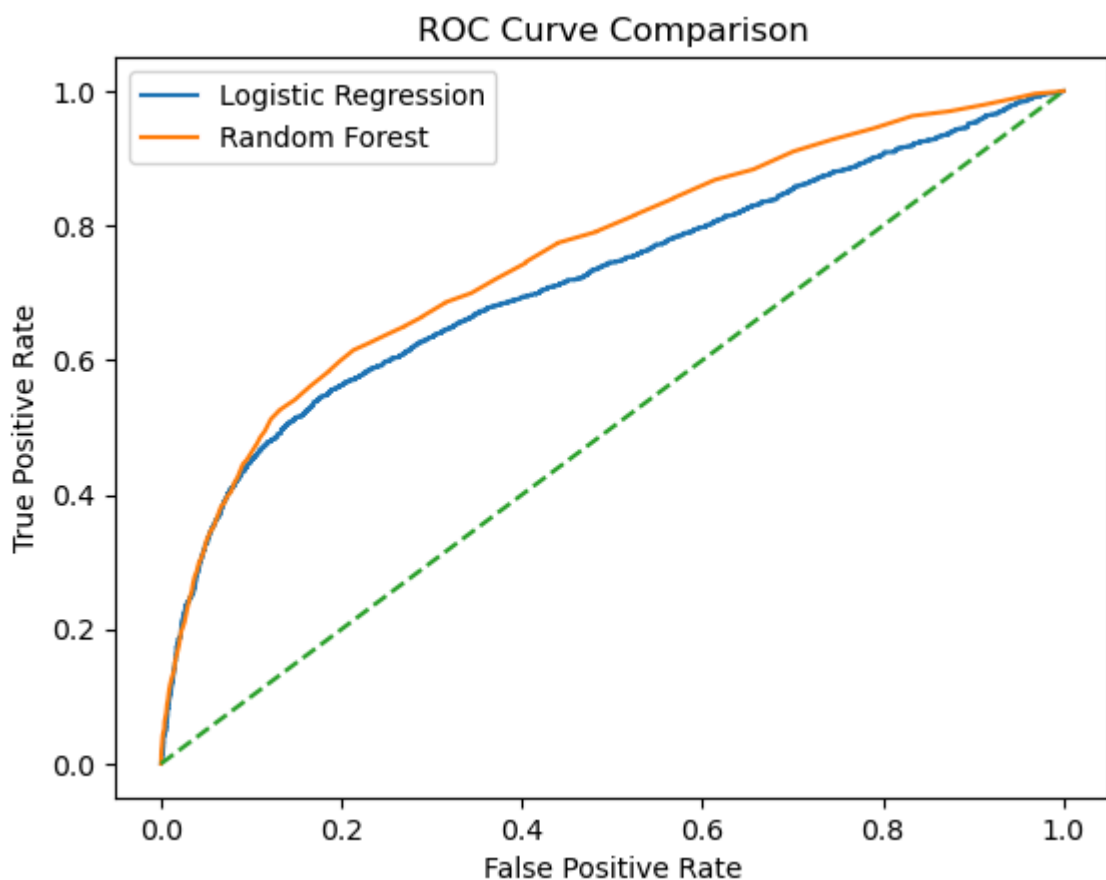
The Random Forest model achieved a ROC-AUC score of approximately 0.76, outperforming the logistic regression model

(AUC ≈ 0.72). This indicates that Random Forest has a stronger ability to distinguish between defaulters and non-defaulters.

```
In [52]: # ROC Curve
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)

plt.plot(fpr, tpr, label="Logistic Regression")
plt.plot(fpr_rf, tpr_rf, label="Random Forest")
plt.plot([0,1],[0,1], '--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.show()
```



The ROC curve comparison shows that the Random Forest model consistently outperforms logistic regression across all classification thresholds. The curve for Random Forest lies above that of logistic regression, indicating superior discriminatory power. This is consistent with the higher ROC-AUC value achieved by Random Forest (≈ 0.76) compared to logistic regression (≈ 0.72).

4.4 Artificial Neural Network

An artificial neural network model was implemented using a multilayer perceptron architecture. The model consisted of two hidden layers with non-linear activation functions, allowing it to capture complex relationships in the data. Feature scaling was essential for this model to ensure efficient training and convergence.

Assumptions for Artificial Neural Network:-

1. Independent Observations
2. Sufficient Data
3. No Severe Imbalance Issues

```
In [53]: df = pd.read_csv("Data - Data.csv", header = 1)
df.head()
```

Out[53]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4
0	1	20000	2	2	1	24	2	2	-1	-1
1	2	120000	2	2	2	26	-1	2	0	0
2	3	90000	2	2	2	34	0	0	0	0
3	4	50000	2	2	1	37	0	0	0	0
4	5	50000	1	2	1	57	-1	0	-1	0

```
In [54]: # Rename Target Variable
df.rename(columns={"default payment next month": "default"}, inplace=True)
```

```
In [55]: # Handle Categorical Variables
df['EDUCATION'] = df['EDUCATION'].replace([0,5,6], 4)
df['EDUCATION'].value_counts()
```

```
Out[55]: EDUCATION
2      14030
1      10585
3       4917
4        468
Name: count, dtype: int64
```

```
In [56]: # Handle Categorical Variables
df['MARRIAGE'] = df['MARRIAGE'].replace(0, 3)
df['MARRIAGE'].value_counts()
```

```
Out[56]: MARRIAGE
2      15964
1      13659
3         377
Name: count, dtype: int64
```

```
In [57]: # Feature Selection
df.drop(columns=['ID'], inplace=True)
```

```
In [58]: # Splitting Dataset
X = df.drop('default', axis=1)
y = df['default']
```

```
In [59]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
# Stratify is important due to class imbalance
```

```
In [60]: # Encoding Categorical Variables
X_train = pd.get_dummies(X_train, columns=['SEX', 'EDUCATION', 'MARRIAGE'],
X_test = pd.get_dummies(X_test, columns=['SEX', 'EDUCATION', 'MARRIAGE'], d
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0
```

```
In [61]: # Feature Scaling
col = ['LIMIT_BAL', 'AGE', 'PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5',
      'PAY_6', 'BILL_AMT1', 'BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4',
      'BILL_AMT5', 'BILL_AMT6', 'PAY_AMT1', 'PAY_AMT2', 'PAY_AMT3',
      'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6', 'SEX_2', 'EDUCATION_2',
      'EDUCATION_3', 'EDUCATION_4', 'MARRIAGE_2', 'MARRIAGE_3']
num_cols = ['LIMIT_BAL', 'AGE'] + [col for col in X_train.columns if 'BILL'
scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

```
In [62]: # Fitting Artificial Neural Network
nn_model = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    activation='relu',
    solver='adam',
    max_iter=200,
    random_state=42
)
nn_model.fit(X_train, y_train)
```

```
/opt/anaconda3/lib/python3.13/site-packages/sklearn/neural_network/_multil
ayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (200) reached and the optimization hasn't converged yet.
warnings.warn(
```

```
Out[62]: ▼ MLPClassifier ⓘ ?
MLPClassifier(hidden_layer_sizes=(64, 32), random_state=42)
```

```
In [63]: # Predictions
y_pred_nn = nn_model.predict(X_test)
y_prob_nn = nn_model.predict_proba(X_test)[:, 1]
```

```
In [64]: # Accuracy
accuracy_score(y_test, y_pred_nn)
```

```
Out[64]: 0.7904444444444444
```

The neural network model achieved an accuracy of approximately 79%, outperforming logistic regression but slightly underperforming compared to the Random Forest model.

```
In [65]: # Confusion Matrix
confusion_matrix(y_test, y_pred_nn)
```

```
Out[65]: array([[6417,  592],
               [1294,  697]])
```

The neural network model demonstrates a balanced trade-off between precision and recall. However, similar to the Random Forest model, it exhibits relatively low recall for defaulters, indicating that a significant number of risky customers are not identified. While its performance is more balanced than Random Forest, it does not outperform logistic regression in detecting default risk.

```
In [66]: # Classification Report
print(classification_report(y_test, y_pred_nn))
```

	precision	recall	f1-score	support
0	0.83	0.92	0.87	7009
1	0.54	0.35	0.42	1991
accuracy			0.79	9000
macro avg	0.69	0.63	0.65	9000
weighted avg	0.77	0.79	0.77	9000

The neural network model demonstrates strong performance in identifying non-defaulters, with high recall for the majority class. However, similar to the Random Forest model, it exhibits relatively low recall for defaulters (approximately 35%), indicating that many risky customers are not identified. Although it achieves a balance between precision and recall, it does not significantly outperform the other models.

```
In [67]: # ROC-AUC
roc_auc_score(y_test, y_prob_nn)
```

```
Out[67]: np.float64(0.7314521495968553)
```

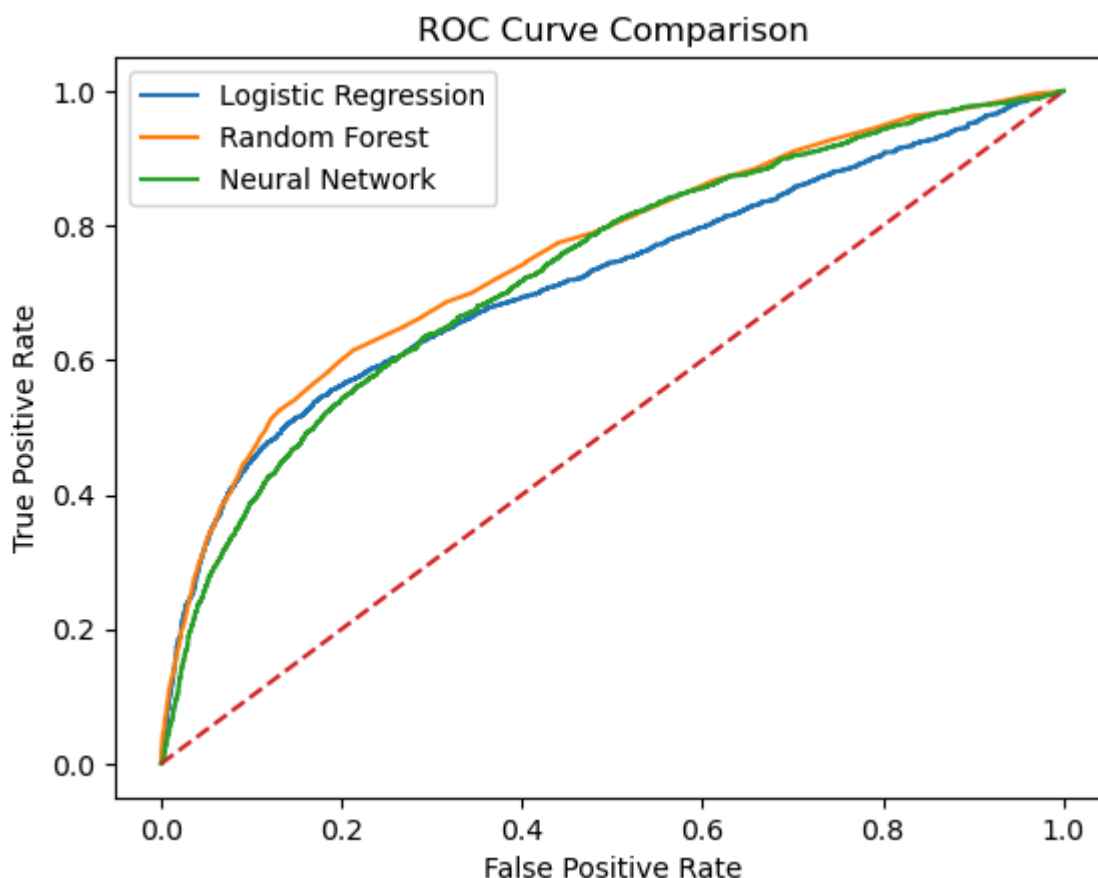
The neural network model achieved a ROC-AUC score of approximately 0.73, indicating a good ability to distinguish between defaulters and non-defaulters. While this represents a slight improvement over logistic regression, it remains lower than the performance of the Random Forest model (AUC $\approx$ 0.76).

```
In [68]: # Roc Curve
fpr_nn, tpr_nn, _ = roc_curve(y_test, y_prob_nn)

plt.plot(fpr, tpr, label="Logistic Regression")
plt.plot(fpr_rf, tpr_rf, label="Random Forest")
plt.plot(fpr_nn, tpr_nn, label="Neural Network")

plt.plot([0,1],[0,1], '--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.show()
```



The neural network performs slightly better than logistic regression, its performance remains close to that of Random Forest without providing a significant improvement.

5. Results

The analysis shows that model choice depends on the trade-off between recall (catching defaulters) and precision (avoiding false alarms).

- If the objective is to minimise risk (e.g. real-world banking), the Logistic Regression model is preferred.
- If the objective is to strengthen overall predictive power, the Random Forest model is preferred.

This study compared logistic regression, Random Forest, and neural network models for credit default prediction. The Random Forest model achieved the highest ROC-AUC, indicating superior ability to distinguish between defaulters and non-defaulters. However, it exhibited low recall for the default class, meaning that many risky customers were not identified. Logistic regression, while less accurate overall, demonstrated significantly higher recall and was more effective in identifying defaulters. The neural network did not significantly outperform the other models despite increased complexity.

From a practical credit risk perspective, where the cost of missed defaults is high, logistic regression may be preferred. However, Random Forest offers strong predictive power and could be further improved through threshold tuning.

6. Conclusion

This study aimed to evaluate and compare the performance of three machine learning models: Logistic Regression, Random Forest, and Neural Networks, for predicting credit default. The analysis demonstrates that model performance depends on the trade-off between recall and precision, which is particularly important in credit risk applications.

The Random Forest model achieved the highest accuracy and ROC-AUC, indicating superior overall predictive performance and strong ability to distinguish between defaulters and non-defaulters. However, it exhibited low recall for the default class, meaning that a significant number of risky customers were not identified. This limitation is critical in a financial context where missed defaults can result in substantial losses.

In contrast, Logistic Regression demonstrated significantly higher recall, making it more effective in identifying defaulters. Although it produced a higher number of false positives, it aligns more closely with the objectives of risk minimisation in real world banking applications. The Neural Network model provided a balance between precision and recall but did not significantly outperform the other models despite increased complexity.

Overall, the findings suggest that Logistic Regression may be preferred when the primary objective is to minimise financial risk, while Random Forest may be more suitable when overall predictive performance is prioritised. These results highlight the importance of aligning model selection with business objectives rather than relying solely on accuracy or ROC-AUC.

6.1 Limitations

This study has several limitations. First, the dataset exhibits class imbalance, which may influence model performance, particularly in detecting defaulters. Second, the neural network model was not extensively tuned, which may limit its performance. Additionally, feature engineering was limited, and more sophisticated transformations could improve predictive accuracy.

6.2 Future Work

Future research could focus on improving model performance through techniques such as threshold tuning, resampling methods (e.g. SMOTE), and hyperparameter optimisation. Incorporating additional features or external data sources may

also enhance predictive power. Furthermore, model interpretability techniques could be explored to better understand the drivers of default risk, which is particularly important in regulated financial environments.

7. References

1. Yeh, I. (2009). *Default of Credit Card Clients [Dataset]*. UCI Machine Learning Repository.
<https://doi.org/10.24432/C55S3H>
2. Yeh, I., & Lien, C. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36, 2473–2480.
3. GeeksforGeeks. (2017, May 9). *Logistic Regression in Machine Learning*. <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>
4. GeeksforGeeks. (2024, February 22). *Random Forest Algorithm in Machine Learning*. <https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>
5. GeeksforGeeks. (2020, June 24). *Artificial Neural Networks and its Applications*. <https://www.geeksforgeeks.org/deep-learning/artificial-neural-networks-and-its-applications/>